常用头文件与库函数速查

下文示例除特别标明外，默认已包含所需头文件，并写了`using namespace std;`；也可以不写这句，在标准库名称前加`std::`。

一、三个常用功能

功能	头文件	名称与用法	注意
int的最大值	`<climits>`	`INT_MAX`，如`int smallest = INT_MAX;`	它是宏常量，不加括号调用；不能对它加1，会发生有符号整数溢出
一行完成排序	`<algorithm>`	`sort(a, a+n);`	数组前n个元素升序排序；范围为左闭右开`[a,a+n)`
设置输出精度	`<iomanip>`	`cout << fixed << setprecision(2) << x;`	保留小数点后两位；名称是`setprecision`，中间没有空格

```
#include <iostream>
#include <climits>
#include <algorithm>
#include <iomanip>
using namespace std;

int main() {
    int smallest = INT_MAX;
    int a[] = {3, 1, 2};
    sort(a, a+3);  // 排序后为1、2、3
    for (int i = 0; i < 3; ++i)
        if (a[i] < smallest) smallest = a[i];
    cout << smallest << '\n';  // 输出1
    cout << fixed << setprecision(2) << 3.14159;  // 输出3.14
    return 0;
}
```

二、输入、输出与字符串

头文件	名称及例子	用途	重点陷阱
`<iostream>`	`cin >> n; cout << n << '\n';`	标准输入输出	`>>`跳过开头空白，字符串读到下一个空白停止
`<iostream>`	`while (cin >> x)`	读到文件结束或提取失败	不写`while (!cin.eof())`，否则可能处理失败的读取

头文件	名称及例子	用途	重点陷阱
`<iomanip>`	`fixed << setprecision(2)`	固定小数点后两位	默认浮点格式下，单独`setprecision(2)`设置两位有效数字；格式会持续生效
`<string>`	`string s; getline(cin,s);`	读整行，保留空格	不保留换行符；空行可得到空串
`<string>`	`s.size()`/`s.length()`	字符串长度	返回无符号类型，空串时不要直接计算`size()-1`再作下标
`<string>`	`s.empty()`	是否为空	先判空，再考虑访问`s[0]`
`<string>`	`s[i]`、`s += c`、`a+b`	下标访问、追加、拼接	有效字符下标是0到长度-1；下标不会自动替你检查范围
`<string>`	`s.substr(pos,len)`	从pos取至多len个字符	第二个参数是长度，不是结束下标；pos不能超过size
`<string>`	`s.find("ab")`	首次出现的位置	找不到是`string::npos`，不是0；结果不要随意转int
`<string>`	`s.erase(pos,len)`/`s.insert(pos,"ab")`	删除/插入字符	修改后长度和后续下标可能改变
`<string>`	`string(k,' ')`	构造k个空格	用于重排空格；k必须非负

头文件	名称及例子	用途	重点陷阱
<limits>	cin.ignore(numeric_limits<streamsize>::max(),'\n')	丢弃当前行剩余部分	用在>>切换到按行读取时；不要对已经开始的有效整行无条件丢弃

两种输入模式不能混淆

```
// 模式A：单词/数字输入，忽略空白。
string name;
cin >> name;  // 输入my god，只读到my

// 模式B：先读数量，再逐行读文本。
int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
for (int i = 0; i < n; ++i) {
    string line;
    getline(cin, line);
    // line保留行首、行内、行末的空格。
}
```

`getline(cin >> ws, s)`会跳过前导空白与空行，所以不适合《重排空格》或需要保留原行内容的题目。`cin.ignore()`不带参数只丢弃一个字符，输入行尾若还有空格便不够稳妥。读到EOF的题直接用`while (getline(cin,line))`。

字符与数字

```
int digit = c - '0';        // 前提：c是'0'到'9'
char ch = char('0' + digit); // 前提：digit是0到9
int index = c - 'a';        // 前提：c是小写英文字母
```

`'7'`是字符，`7`是整数，`"7"`是字符串字面量。统计26个英文字母之前一定先判断字符范围，不能拿标点直接当数组下标。

三、字符分类：<cctype>

名称	含义	例子与注意
isdigit(c)	是否为数字字符	判断'0'到'9'
isalpha(c)	是否为字母	依赖当前区域设置；基础英文题也可直接检查A–Z、a–z
isalnum(c)	是否为字母或数字	不包含下划线
islower(c) / isupper(c)	是否为小写 / 大写字母	适合密码类别标记

名称	含义	例子与注意
`isspace(c)`	是否为空白	不只有普通空格，还包括换行、制表符等
`tolower(c)` / `toupper(c)`	转小写 / 大写	返回转换后的值，不会自动修改原变量

分类函数返回0或非0，不保证真值恰好等于1。对可能为负值的普通char，正确调用形式是：

```
unsigned char uc = static_cast<unsigned char>(c);
if (isdigit(uc)) { /* 是数字 */ }
c = static_cast<char>(tolower(uc));
```

基础英文题可直接检查字符范围，避免依赖区域设置。上述函数按字节处理，不用于识别中文汉字。

四、C风格字符串：<cstring>

若课程主要使用`char[]`，需额外复习下表。`std::string`与`char[]`是两套接口，不要混用。

名称	含义	示例与限制
`strlen(s)`	终止零之前的字符数	不包含`'\0'`；数组必须已正确终止
`strcmp(a,b)`	按字典序比较	`<0`、`==0`、`>0`分别代表小于、相等、大于；不是只返回-1/0/1
`strcpy(dst,src)`	复制，包含终止零	dst容量至少为strlen(src)+1
`strcat(dst,src)`	追加，包含终止零	dst须已有合法字符串，且容量足够
`strchr(s,c)`	找首次出现的字符	返回指针，没找到为`nullptr`；若指针未学，可仅识别用途
`strstr(s,t)`	找首次出现的子串	返回指针，没找到为`nullptr`
`memset(a,0,sizeof a)`	把存储区域的字节清零	不能用`memset(a,1,...)`把int数组每个元素设为1；通用赋值用循环或fill

```
char s[81];        // 最多80个字符，还要给'\0'留1个位置
cin.getline(s,81); // 最多存入80个字符加终止零；行过长会设置失败状态
```

C字符串比较内容用`strcmp(a,b)==0`，不能用数组名`a==b`比较内容。`std::string`则可以直接用`a==b`。不要使用已淘汰且无法控制长度的`gets`。

五、数学、排序与初始化

头文件	名称	用法	易错点
`<climits>`	`INT_MAX`、`INT_MIN`	int的最大值、最小值	数值依赖实现；不要假设所有整数类型的范围相同

头文件	名称	用法	易错点
`<climits>`	`LLONG_MAX`、`LLONG_MIN`	long long的最大值、最小值（C++11）	超出对应类型范围会溢出
`<cmath>`	`sqrt(x)`	平方根，例如`sqrt(a*a+b*b)`	x应非负；若a、b是大整数，乘法先溢出再转double仍错误
`<cmath>`	`fabs(x)`	浮点绝对值	整数也可用`<cstdlib>`中的abs；最小负整数的绝对值可能超出原类型范围
`<cmath>`	`floor(x)`、`ceil(x)`	向下 / 向上取整	`floor(-1.2)=-2`，`ceil(-1.2)=-1`；不同于整数转换向零截断
`<cmath>`	`round(x)`	最接近整数，正好一半时远离0	浮点表示误差仍要考虑
`<cmath>`	`pow(x,y)`	幂	用于浮点计算；整数平方优先x*x，不把pow当精确整数运算
`<algorithm>`	`min(a,b)`、`max(a,b)`	两数最值	两个实参类型应匹配
`<algorithm>`	`sort(a,a+n)`	数组升序排序	区间左闭右开；不保证相等项稳定
`<algorithm>`、`<functional>`	`sort(a,a+n,greater<int>())`	int数组降序排序	greater位于`<functional>`；也可升序排序后reverse
`<algorithm>`	`stable_sort(a,a+n)`	稳定排序	等价项保留原顺序；按关联分数排序还需把分数与原数据关联
`<algorithm>`	`reverse(s.begin(),s.end())`	反转字符串	会修改原串；迭代器未学则用下标交换即可
`<algorithm>`	`fill(a,a+n,-1)`	给每个数组元素赋-1	按元素赋值，区别于memset按字节设置
`<utility>`	`swap(a,b)`	交换两个对象	关联数组必须一起交换
`<string>`	`stoi(s)` / `stoll(s)`	字符串转int / long long（C++11）	超范围或无法转换会抛异常；可能只转换有效数字前缀，不适用于200位大整数

排序算法、迭代器与自定义比较规则是否需要掌握，以授课教师要求为准。

六、若课堂使用printf/scanf

头文件为`<cstdio>`。可选择C风格或C++风格输入输出，初学时同一程序尽量保持一种风格。

操作	示例	注意
读int	`scanf("%d", &n)`	需传地址，变量前的&不能省
读double	`scanf("%lf", &x)`	scanf的double用%lf，float用%f
输出两位小数	`printf("%.2f\n", x)`	printf输出double用%f
读long long	`scanf("%lld", &v)`	输出也用%lld
读字符	`scanf(" %c", &c)`	前导空格表示跳过空白；需要读空格本身时不能这样写
读无空格短串	`scanf("%80s", s)`	配合char s[81]；数组名不加&，限制宽度防越界

七、考前一页检查表

1. 输入有几组：给数量、读到EOF，还是遇到哨兵结束？
2. 字符串有没有空格：`cin >> s`还是`getline`？
3. 下标从0还是从1开始？数组容量够不够？
4. 每组数据、每个元素的计数器是否重新初始化？
5. `=`与`==`、`&&`与`||`是否写对？数学链式比较要写成`2<=x && x<=9`。
6. 整数除法是否需要小数？需要时先转浮点，如`1.0*a/b`；不要等a/b已截断后才赋给double。
7. `int a[26] = {-1};`只有第一个元素是-1，余下为0；全设-1请用循环或fill。
8. 最大规模下循环次数是否合理？整数乘法与累加会不会溢出？
9. 是否测试空数据、单元素、全相等、首尾边界、恰好截止，以及多组间状态重置？
10. 输出大小写、小数位、空格、标点与换行是否符合题面？不要附加“答案为”等文字。